

Mobile Networks and IoT [2026-2027]

Project 01 – Smart Environment Monitoring with Network Failure and Recovery Analysis

Introduction

This project aims to create a full IoT solution to collect, process and visualise Wi-Fi data and reliability information from a monitoring system. Students will have to program MQTT publishers and subscribers, process the data gathered, store it in a database, and finally visualize it on a dashboard. The richness of the info gathered and displayed is highly valued. Students should demonstrate the complete pipeline, test alerts, explain the main findings, and submit a short technical report.

Objectives

Students successfully concluding this project should be able to:

- Produce and collect real-time data
- Understand the effects of the quality of Wi-Fi connection on data transmission
- Transfer data using MQTT
- Process data coming from sensors and save it in a database
- Display and analyse real-time and historical data in a dashboard
- Evaluate network behaviour

Support Material

- MQTT broker running on a Raspberry Pi available at the Department of Informatics Engineering
- All documents and examples from previous classes
- Python JSON: https://www.w3schools.com/python/python_json.asp
- Linux `iwconfig` command: https://linux.die.net/man/8/iwconfig
- Linux `iw` command: https://linux.die.net/man/8/iw
- Linux `iwlist` command: https://linux.die.net/man/8/iwlist

Description of work

Scenario

This project will create a **smart environment monitoring system over an IoT network**. Monitoring devices may be distributed across various locations within the organization's facilities and will report Wi-Fi performance data to the network administrator responsible for managing the company's wireless infrastructure. Although the number of devices available for this exercise is limited, the system dashboard will be designed to support monitoring of at least three devices simultaneously, ensuring scalability for future expansion. The primary objective is to enable **real-time Wi-Fi management and monitoring**. To analyze the network behaviour, controlled tests over Wi-Fi should be done - brief connectivity interruptions, measure missing or delayed data, investigate whether measurements are recovered after reconnection, and describe possible security problems. MQTT retained messages, QoS levels or authentication can be used to limit some of the problems detected.

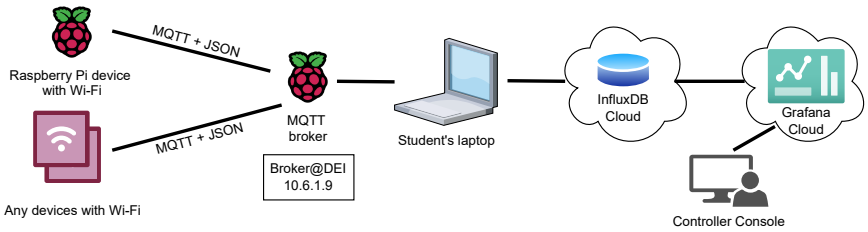


Figure 1 – Generic setup

Project Requirements

- A Raspberry Pi simulates the collection of environmental data (that can be randomly generated) at regular intervals and sends it to a database;
- The Raspberry Pi also collects Wi-Fi data at regular intervals;
- Data is formatted as **JSON objects** and transmitted via MQTT;
- An MQTT broker handles communication between the Raspberry Pi and the PC;
- A PC-side script receives the MQTT data, performs pre-processing (if needed), and writes the results to InfluxDB;
- All collected data will be stored in a **time-series database - InfluxDB**, enabling continuous data access, trend analysis, and performance evaluation over time using a **visualization tool – Grafana**;
- A Grafana dashboard visualizes key metrics such as Wi-Fi signal strength, ping, SSID, and other relevant parameters;
- Additional Grafana dashboards also analyze the effect of network connectivity on data collection;
- The dashboard should present evidence that the system works reliably;
- Alerts are configured in Grafana to notify of low reliability;
- The system supports monitoring of up to 3 Wi-Fi devices simultaneously;

Example of data to collect

The information to collect may include data such as:

- Quality of RSSI over time;
- Devices per Wi-Fi channel;
- Ping latency (to dns.google, IP=8.8.8.8);
- Connection uptime;
- SSIDs in use;
- Wi-Fi channels in use (you may have to process the available data to get the channel number);
- Any data simulating the environmental sensors.

The richness and relevance of the data selected will be highly valued.

MQTT Broker

A Raspberry Pi with a Mosquitto v2.0.11 MQTT broker, located at IP 10.6.1.10, port 1883, will be available for students. This broker will manage all the students' topics.

To prevent topic conflicts, use the following topic format::

```
[your email user at student.dei.uc.pt]/[any hierarchy of topics needed]
```

Storage

An InfluxDB database will be used to store data. This database must be created in InfluxDB Cloud (https://www.influxdata.com/). The registration of a free plan is required (registration has already been made in previous classes, students can use the same one). The database should hold all the necessary values to enable the following analysis of the network.

Control Dashboard

Grafana will be used to create a control dashboard. This dashboard must be created in Grafana Cloud (https://grafana.com). The registration of a free plan is required (registration has already been made in previous classes, students can use the same one).

The data analysis should include two separate sections:

Section 1: Summary dashboard

- Students should select the most important information to display on the dashboard and choose the most appropriate Grafana charts and graphs. Information about the Wi-Fi status should be included. The dashboard should be designed in a way that allows an operator to easily understand the reliability of the system in real-time. Each student must establish their own criteria for displaying the most relevant and essential information.

Section 2: Detailed dashboard

- This dashboard should allow users to analyse each device in greater detail and review its historical values.

Software to use

- Python programming language (version 3.7+);

- Mosquitto software (version 2.0+);
- InfluxDB Cloud;
- Grafana Cloud;
- Linux running on the Raspberry Pi;
- Windows/MacOS/Linux for the PC.

Working at home with no Raspberry Pi available

For test purposes out of classes, students without a Raspberry Pi may access the broker at DEI using the DEI VPN. For more information on how to set-up the VPN, access the DEI helpdesk website at: https://helpdesk.dei.uc.pt .
Students may also generate data from their own PCs Wi-Fi.

Project delivery

This section describes the delivery conditions and submission policy for the project.

Delivery conditions

The project must be completed **in groups of a maximum of 2 students**.

Project submission

The final **zip** file must be submitted to Inforestudante. **Submissions via email will not be accepted!**

- Contents of the zip file:
  - Python code
  - Configuration files (if needed)
  - Documentation (pdf) – max 15 pages
    - Architecture/diagram
    - JSON structure
    - Database structure
    - Screenshots of Grafana dashboards
    - Alert configurations
    - Engineering logbook tracking:
      - Process, decisions and failures (initial diagrams, errors encountered, debugging needed, design decisions, AI prompts used, tests performed);
    - AI use declaration
      - Which tools were used;
      - Where the tools were used (research/code generation/debugging/Report/Diagrams/...);
    - Validation tests performed;
    - Comparison of design choices and limitations discussion;
  - Put your name in all the documents

Submission date: 02/November/2026
Defence date: Weeks of 02 and 09/November/2026

Evaluation

The evaluation will consist of the categories listed in Table 1, weighted according to each student's defence performance. Not being present for the defence will result in a grade of 0 for that student. During the oral defense and live demo, students must understand all the submitted code, all the diagrams presented, all the configurations made, and be able to explain and modify them during the oral defense, including modifying any part of the project or creating different graphics.

Table 1. Evaluation criteria

Module	Value
Data ingestion	20%
Transmission of data using MQTT	10%
Storage	10%
Control Dashboard and quality of analysis	20%
Report	40%
Total	100%

Plagiarism

There will be no tolerance for plagiarism or any other type of fraud. Attempts to do so will be graded ZERO and will imply the failure of the course. Depending on the severity, the situation may lead to disciplinary proceedings. Code will be subject to inspection and comparison between all students.

Annex

Testing values

For testing purposes, a Raspberry Pi is sending data every 10 seconds to the MQTT Broker at 10.6.1.10 on topic `/rmic/wifi/metrics`. The data is formatted as a JSON (JavaScript Object Notation) object with the following structure (specific values are just an example):

```
{
  'ssid': 'IoT-test',
  'bssid': '06:ec:da:aa:47:5e',
  'freq_mhz': 2462,
  'channel': 11,
  'signal_dbm': -56,
  'tx_bitrate_mbps': 24.0,
  'link_quality': 77.1,
  'ping_ms': 17.7,
  'ip_addr': '100.100.100.100',
  'hostname': 'raspberrypi',
  'timestamp': 1761848413.048363,
  ‘temperature’: 23,
  ‘humidity’: 35
}
```

The ping value represents the latency to IP address 10.16.0.254. For privacy and testing purposes, the IP address shown is a dummy value. All other values are real.

Available Raspberry for tests

A Raspberry Pi is provided for you to test and run your Wi-Fi and sensor data collection (or synthetically generated) scripts (only these scripts!). It is located at IP address 10.6.1.79 with user = `rpiuser` and password = `rpipass`.

When using the Raspberry Pi, create your own directory and do not create any files outside it. Delete your files before you log out, as the Raspberry Pi is shared and accessible to everyone.

Ensure that none of your files remain in the filesystem and none of your processes are left running when you log out.

Do not install anything on the Raspberry Pi.